

# Sprawozdanie

**Nr laboratorium:** 3

**Temat:** Interpolacja funkcji

**Autor:** Vladyslav Humeniuk

**Data:** 15.11.2025

---

## 1 Co było do zrobienia

- Zaimplementować program w języku C wykorzystujący bibliotekę **GSL** (GNU Scientific Library) do interpolacji funkcji:

$$f(x) = \frac{1}{1 + 25x^2}$$

w przedziale  $x \in [-1, 1]$ .

- Przeprowadzić interpolację, wykorzystując 11 węzłów równoodległych, trzema metodami:
    1. **Wielomianową** (`gsl_interp_polynomial`).
    2. **Funkcjami sklejanymi 3. stopnia** (`gsl_interp_cspline`).
    3. **Liniovą** (`gsl_interp_linear`).
  - Wygenerować wykresy porównawcze w programie **Gnuplot**, prezentujące funkcję analityczną, węzły oraz wyniki poszczególnych interpolacji.
  - Zidentyfikować problem występujący przy interpolacji wielomianowej (efekt Rungego) i wyjaśnić jego przyczyny.
- 

## 2 Moje podejście do rozwiązania problemu

Do realizacji zadania wykorzystano bibliotekę numeryczną GSL, która oferuje gotowe procedury interpolacyjne. Program `interpolacja_m.c` został podzielony na etapy:

1. **Generowanie węzłów:** Obliczono wartości funkcji  $f(x)$  dla 11 punktów ( $\text{step} = 10$ ) rozmieszczonych równomiernie w przedziale  $[-1, 1]$ .
2. **Interpolacja:** Utworzono funkcję pomocniczą `make_interpolation`, która przyjmuje typ interpolacji jako argument. Pozwoliło to na czyste i modułowe wygenerowanie danych dla trzech

różnych metod bez powielania kodu.

3. **Próbkowanie zagęszczone:** Aby wykresy interpolacji były gładkie, obliczono ich wartości z krokiem `0.01` (znacznie gęściej niż węzły).

Do wizualizacji użyto skryptu `plots.gp`, który wykorzystuje tryb `multiplot` do czytelnego zestawienia trzech metod obok siebie na jednym obrazku.

kod z pliku `interpolacja_m.c`:

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <gsl/gsl_errno.h>
#include <gsl/gsl_spline.h>
#include <gsl/gsl_interp.h>

static double fun(double x)
{
    return 1.0 / (1.0 + 25.0 * x * x);
}

void make_interpolation(const gsl_interp_type *T, const char *filename,
                       double *x, double *y, int steps, double a, double b)
{
    gsl_interp_accel *acc = gsl_interp_accel_alloc();
    gsl_spline *spline = gsl_spline_alloc(T, steps + 1);

    FILE *output = fopen(filename, "w");
    double xi, yi;

    gsl_spline_init(spline, x, y, steps + 1);

    for (xi = a; xi <= b; xi += 0.01)
    {
        yi = gsl_spline_eval(spline, xi, acc);
        fprintf(output, "%g %g\n", xi, yi);
    }

    fclose(output);
    gsl_spline_free(spline);
    gsl_interp_accel_free(acc);
}

int main (void)
{
    const double a = -1.0;
    const double b = 1.0;
    const int steps = 10;

    double x[100], y[100], dx;
```

```

FILE *input;
int i;

input = fopen("values.txt", "w");
dx = (b - a) / (double) steps;

for (i = 0; i <= steps; ++i)
{
    x[i] = a + (double)i * dx;
    y[i] = fun(x[i]);
    fprintf(input, "%g %g\n", x[i], y[i]);
}
fclose(input);

/* 3 different interpolation */

/* Polynomial */
make_interpolation(gsl_interp_polynomial, "inter_poly.txt", x, y, steps, a, b);

/* Cubic Spline */
make_interpolation(gsl_interp_cspline, "inter_cspline.txt", x, y, steps, a, b);

/* Linear */
make_interpolation(gsl_interp_linear, "inter_linear.txt", x, y, steps, a, b);

printf("values.txt, inter_poly.txt, inter_cspline.txt, inter_linear.txt\n");

return 0;
}

```

kod z pliku plots.gp:

```

set terminal jpeg size 1600,500
set output "out.jpg"

set grid

set xlabel "X"
set ylabel "Y"

set xrange [-1:1]
set yrange [-1:2]

# Funkcja analityczna (wzór)
f(x) = 1.0 / (1.0 + 25.0 * x * x)

# labels
l0 = "Analytical function 1/(1+25x^2)"
l1 = "Multivariable Interpolation"
l2 = "Nodes"

```

```

l3 = "CSpline"
l4 = "Linear"
set label
set key left

set multiplot layout 1,3 columns

plot f(x) with lines lw 2 lt rgb "black" t l0, \
    "inter_poly.txt" with lines lw 2 lt rgb "red" t l1, \
    "values.txt" with points pointtype 7 pointsize 1.5 lc rgb "black" t l2

plot f(x) with lines lw 2 lt rgb "black" t l0, \
    "inter_cspline.txt" with lines lw 2 lt rgb "blue" t l3, \
    "values.txt" with points pointtype 7 pointsize 1.5 lc rgb "black" t l2

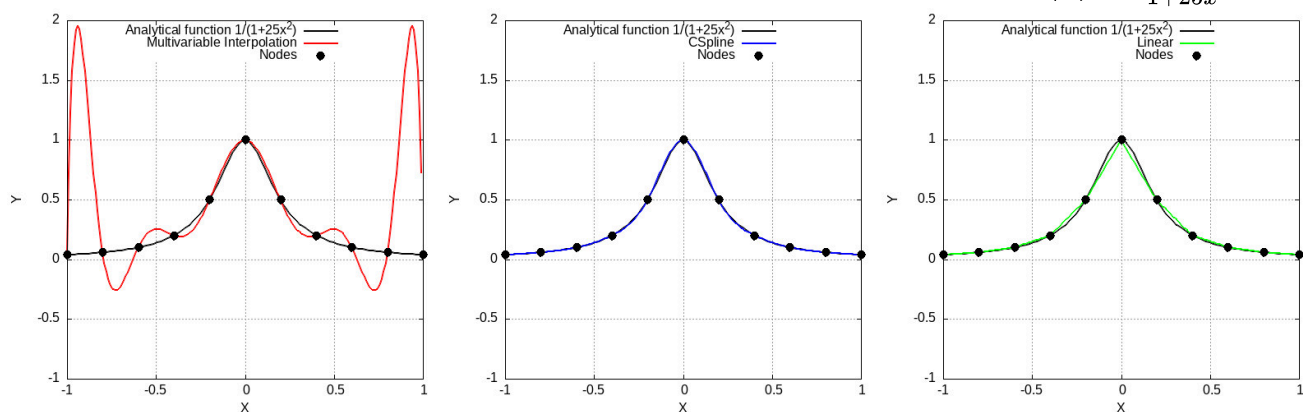
plot f(x) with lines lw 2 lt rgb "black" t l0, \
    "inter_linear.txt" with lines lw 2 lt rgb "green" t l4, \
    "values.txt" with points pointtype 7 pointsize 1.5 lc rgb "black" t l2

unset multiplot

```

### 3 Wyniki

Poniższy wykres przedstawia porównanie trzech metod interpolacji dla funkcji  $f(x) = \frac{1}{1+25x^2}$ .



#### Analiza wykresów:

- Interpolacja wielomianowa (czerwona):** W środkowej części wykresu pokrywa się z funkcją, ale na krańcach przedziału (blisko -1 i 1) występują potężne oscylacje. Wartość interpolowana drastycznie odbiega od rzeczywistej.
- CSpline (niebieska):** Bardzo dobrze przybliża funkcję oryginalną. Krzywa jest gładka i nie wykazuje oscylacji.
- Linowa (zielona):** Jest stabilna (brak oscylacji), ale "kanciasta". W miejscach dużej krzywizny funkcji (okolice  $x = 0$ ) błąd przybliżenia jest widoczny, choć przewidywalny.

## 4 Wnioski

1. **Efekt Rungego:** Obserwowane pogorszenie jakości interpolacji wielomianowej na krańcach przedziału nosi nazwę **efektu Rungego**.
  2. **Przyczyny zjawiska:** Efekt ten powstaje przy interpolacji wielomianami **wysokiego stopnia** (w tym przypadku stopnia 10, ponieważ mamy 11 węzłów) przy użyciu **węzłów równoodległych**. Mimo że wielomian przechodzi idealnie przez wszystkie zadane punkty, pomiędzy nimi (szczególnie na brzegach dziedziny) zaczyna gwałtownie oscylować. Zwiększanie liczby węzłów w tym przypadku nie poprawia wyniku, lecz go pogarsza (błąd dąży do nieskończoności).
  3. **Rozwiązanie:** Aby uniknąć tego efektu, w zastosowaniach inżynierskich stosuje się **funkcje sklepane** (splines), które łączą wielomiany niskiego stopnia (zazwyczaj 3.) na poszczególnych podprzedziałach. Jak widać na wykresie środkowym (`CSpline`), metoda ta daje najlepsze rezultaty dla gładkich funkcji.
  4. Alternatywnym rozwiązaniem dla wielomianów wysokiego stopnia byłoby zastosowanie tzw. [Chebyshev nodes](#) (zagęszczonych na krańcach), co zminimalizowałoby błąd interpolacji, [więcej](#)
- 

## 5 Bibliografia

1. [GSL Documentation: Interpolation.](#)
2. [Runge's phenomenon - Wikipedia](#)
3. [Chebyshev nodes - Wikipedia](#)